

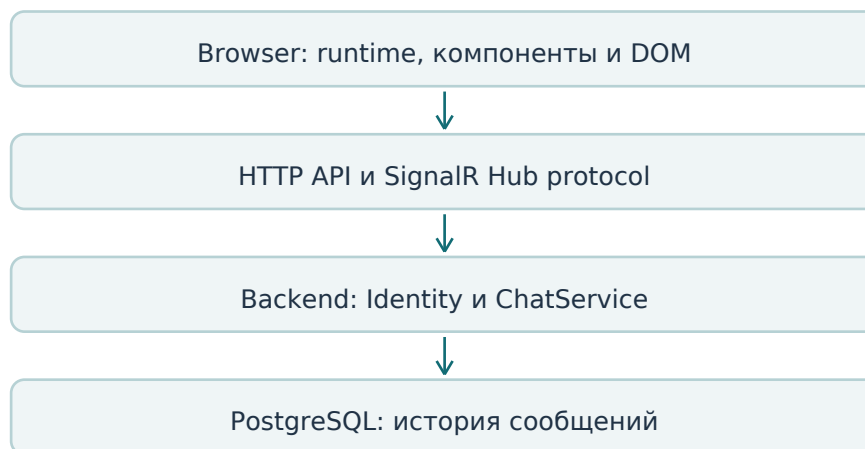
МЕХАНИЗМЫ ВМЕСТО ЗАКЛИНАНИЙ

Blazor WebAssembly

Как это работает

От загрузки browser-runtime до HTTP API, cookie, SignalR и восстановления сообщений после обрыва связи.

Последовательный учебник на примере WASM-этапа Real-Time Chat. Как C# работает в браузере и взаимодействует с backend.



.NET 10 • 40 страниц • объяснения, схемы и практика

Подготовлено для review проекта Real-Time Chat

Содержание

Как читать и с чем сравнивать	3
01-04. Клиент, bootstrap, runtime и AOT	4
05-08. Песочница, async и компоненты	8
09-13. События, Router, hosting и DI	12
14-18. HTTP, JSON, cookie и backend	17
19-23. HubConnection, протокол и группы	22
24-29. История, reconnect, typing и interop	27
30-32. Производительность, выбор и deploy	33
Практика 1: локальное и сетевое действие	36
Практика 2: reconnect и гонки	37
Опорные ответы для самопроверки	38
Источники и дальнейшее чтение	39

Для разбора HubConnectionBuilder: разделы 19-25. Для cookie и различий с Server: разделы 14-17 и 31. Полный маршрут: читать последовательно и проверять наблюдения.

Как читать и с чем сравнивать

Второй учебник: те же действия, другое место исполнения.

Этот текст продолжает учебник о Blazor Server, но его можно читать отдельно. Сквозной пример - наша готовая версия RealTimeChat.BlazorWasm, сохранённая в Git под тегом dotnet-chat-wasm-v4.0.0. Комнаты и история загружаются через HTTP API, а сообщения, присутствие и typing идут через SignalR.

Основной вопрос каждой страницы: где выполняется данный участок кода и какую границу он пересекает? Если C# находится в браузере, это не означает, что PostgreSQL, Identity и ChatService тоже туда переехали. Если сервер раздаёт клиентские файлы, это не означает, что он исполняет Room.razor клиента.

В нашей реализации клиент - отдельное standalone WASM-приложение, которое затем размещено вместе с backend по одному адресу. Это отличается от Blazor Web App с Interactive WebAssembly и prerendering. Здесь нет server-circuit для работы компонентов клиента. Не смешивай эти модели из-за одинакового слова Blazor.

Как работать с текстом

Читай по 3-5 разделов. Закрой материал и восстанови один сценарий: загрузка страницы, вход, JoinRoom, отправка, приём сообщения или reconnect. Затем проверь схему через DevTools и debugger. Повтори восстановление на следующий день.

Фрагменты показывают механизмы и отдельные части нашего кода. Это не инструкция вставить все примеры подряд. Готовое приложение дополнительно требует проверки ошибок, доступа, отмены и освобождения ресурсов. Известные ограничения учебной версии отдельно названы.

Текст и примеры авторские, ссылки ведут к официальной документации и browser API. Ориентир - .NET 10. Для сравнения с первым учебником следи за тремя вещами: где находится компонент, кто выполняет HttpClient и что именно передаётся по realtime-соединению.

Вернуться по памяти: До чтения: почему HttpClient из WASM может выдать cookie браузеру через ответ API, а HttpClient из Blazor Server не делает это автоматически?

01. Клиент действительно живёт в браузере

Два процесса не превращаются в один из-за общего языка.

В `RealTimeChat.BlazorWasm` экземпляр `Room.razor` выполняется на стороне браузера. Поля `_messages`, `_draft`, `_recipientId` и `_hubConnection` находятся в клиентском `.NET runtime`. На сервере живут `ChatService`, репозитории, `EF Core`, `Identity` и `ChatHub`. Это разные объекты в разных средах.

Одинаковый `C#` по обе стороны облегчает совместное использование DTO и привычного синтаксиса. Но обычный вызов метода не может перескочить из браузера в серверный процесс. Для чтения истории нужен HTTP-запрос. Для вызова `SendMessage` нужен протокол `SignalR`.

В браузере	На backend
<code>Room.razor</code> и <code>Home.razor</code>	ASP.NET Core и endpoints
HttpClient клиента, <code>HubConnection</code>	<code>ChatHub</code> и <code>ChatService</code>
DOM и клиентские сборки	Identity, репозитории, PostgreSQL

Даже если сайт открывается по адресу `localhost` и всё физически на одном компьютере, архитектурная граница остаётся. Браузерная песочница не даёт `C#` клиента напрямую читать память backend или открыть произвольное подключение к базе.

При двух вкладках каждая загружает и запускает свой экземпляр приложения со своим UI-состоянием. Cookie общего сайта могут быть общими в профиле браузера, но список сообщений и черновик компонента не становятся автоматически одним объектом. Для общего локального хранения нужен отдельный механизм.

Когда мы разместили WASM и API в одном хосте, мы упростили адреса и cookie. Мы не перенесли исполнение компонентов на сервер. Хост лишь доставляет файлы, а затем обслуживает API и Hub.

Вернуться по памяти: Почему внедрить `ChatService`, который использует серверный `DbContext`, в WASM-компонент недостаточно для доступа к PostgreSQL?

Для проверки деталей: [1], [2]. Ссылки на с. 39-40.

02. Открытие сайта: цепочка загрузки

Чтобы выполнить C#, браузер сначала получает среду исполнения.

Полная загрузка начинается с запроса документа, обычно `index.html`. В нём есть элемент для корневого UI, CSS и bootstrap-скрипт Blazor. Скрипт запускает загрузку runtime и ресурсов приложения. Browser не исполняет `Program.cs` прямо из текстового файла.



Сведения о нужных ресурсах создаются сборкой SDK. Конкретный набор файлов и формат bootstrap-метаданных зависит от версии .NET и публикации. Не нужно считать один исторический файл `blazor.boot.json` обязательной неизменной частью всех версий. В .NET 10 данные boot-манифеста перенесены из отдельного `blazor.boot.json` в `dotnet.js`.

Runtime должен быть готов до выполнения клиентской логики. Поэтому первая загрузка может показывать индикатор. Доступный размер загрузки зависит от сборок, зависимостей, сжатия и кэширования. Это другой компромисс, чем небольшой initial HTML и серверное UI-соединение в Blazor Server.

Когда bootstrap завершён, создаётся root-компонент App, Router выбирает страницу и начинается её lifecycle. Только затем Home запрашивает `api/auth/me` и комнаты. Эти запросы не нужны для самой загрузки языка C#, но нужны для данных нашего приложения.

В DevTools раздельно видны ресурсы framework, сборки, запросы API и установление Hub-соединения. Если приложение не стартует, сначала определи, на каком из этих этапов оно остановилось.

Вернуться по памяти: Почему ошибка загрузки runtime и ответ 401 от `api/auth/me` относятся к разным этапам?

Для проверки деталей: [2], [3], [31]. Ссылки на с. 39-40.

03. WebAssembly, .NET и C#

Браузер не получил встроенное понимание языка C#.

WebAssembly - формат кода и модель выполнения, которую поддерживает браузер. Он не является вариантом HTML и не заменяет DOM. Браузер исполняет wasm-модули, а .NET предоставляет среду, в которой работают managed-код, объекты и стандартные библиотеки.

Обычная сборка C# сначала превращается в .NET intermediate language, или IL. Для базового режима Blazor WASM используется .NET runtime, реализованный для WebAssembly: он может интерпретировать IL, выполнять managed-операции и предоставлять runtime-возможности. Поэтому объяснение «каждый C#-метод просто сразу скомпилировали в wasm» неверно для стандартного режима.

Слой	Что делает
C# / Razor	Исходный код приложения
.NET IL и metadata	Результат обычной managed-сборки
.NET runtime для WebAssembly	Исполнение managed-кода
Browser WebAssembly engine	Исполнение wasm-модулей
JavaScript и DOM	Интеграция с документом браузера

Для доступа к DOM и browser API нужна интеграция с JavaScript. Blazor берёт эту связку на себя для рендеринга. Когда мы пишем `@onclick` и меняем список сообщений, C# работает локально, но DOM всё равно меняется через браузерную инфраструктуру.

Формат упаковки клиентских сборок тоже не обязан быть привычным .dll. Современный SDK может использовать Webcil, упаковывая managed-assembly в WebAssembly-контейнер. Расширение .wasm у ресурса ещё не доказывает, что вся логика внутри AOT-компилирована.

Вернуться по памяти: Назови разницу между wasm-runtime и IL нашего приложения. Почему оба могут быть нужны при запуске?

Для проверки деталей: [1], [4], [5]. Ссылки на с. 39-40.

04. Интерпретация и AOT

Быстрее исполнение не обязательно быстрее первая загрузка.

Без включённого AOT Blazor WebAssembly выполняет IL через runtime с интерпретатором и дополнительными оптимизациями, включая частичную JIT-поддержку `jitterpreter`. Это не обычный серверный CoreCLR JIT, который свободно генерирует native-код для системы. Среда браузера имеет другие ограничения.

AOT, ahead-of-time compilation, заранее компилирует managed-код в WebAssembly. Это может ускорить интенсивные вычисления. Цена - более дорогая публикация и часто больший объём клиентских ресурсов. Из одного слова AOT не следует, что весь сайт станет быстрее во всех сценариях.

```
<PropertyGroup>
  <RunAOTCompilation>true</RunAOTCompilation>
</PropertyGroup>
```

Это пример настройки публикации, а не рекомендация включить её сейчас. Нужны соответствующие WebAssembly build tools. Обычный запуск разработки не заменяет измерение опубликованной Release-версии.

Наш чат большую часть времени ждёт пользователя, API или сообщение Hub. Если узкое место - сеть и база, ускорение локального исполнения IL мало изменит общую задержку. Календарь с тяжёлыми вычислениями или таблица с формулами может получить больше пользы.

Оценивать нужно два разных времени: до готовности приложения при холодной загрузке и время конкретного действия после старта. Уменьшив второе, можно увеличить первое. Кэш браузера меняет повторную загрузку, но не гарантирует, что каждый новый посетитель уже имеет bundle.

WASM не означает автоматически «быстрее JavaScript». Производительность зависит от алгоритма, runtime, interop, DOM и объёма передаваемых данных.

Вернуться по памяти: Почему AOT может почти не ускорить получение двадцати сообщений из удалённой PostgreSQL?

Для проверки деталей: [4], [6]. Ссылки на с. 39-40.

05. Песочница и доверие к клиенту

Код пользователя не является границей безопасности.

Браузер запускает приложение в ограниченной среде. WASM не даёт ему произвольный доступ к диску, чужим сайтам и серверной базе. Используются browser API, запросы и разрешения. Поэтому добавление Npgsql в клиент не создаёт обычный серверный канал к PostgreSQL.

При этом пользователь контролирует своё устройство и клиентское приложение. Он может исследовать сборки и сетевые сообщения, изменять состояние, подменять запросы. Нельзя хранить в bundle пароль PostgreSQL, приватный API-key или правила, на которых единственно держится авторизация.

Клиентская проверка длины сообщения удобна для UX. Серверная проверка обязательна для корректности. Наш `textarea maxlength="500"` и проверка `text.Length` в `SendAsync` помогают пользователю, но атакующий может обратиться к Hub независимо от этой формы.

Сервер получает автора из проверенной идентичности соединения, а не из клиентского поля `authorId`. Проверяет комнату, доступ и существование получателя. Репозиторий фильтрует историю приватных сообщений по пользователю. Скрытый элемент UI не защищает данные, уже отправленные клиенту.

Клиент делает для удобства	Сервер обеспечивает
Показывает доступные комнаты	Право читать нужную комнату
Ограничивает длину в форме	Допустимость текста и записи
Скрывает чужие private сообщения	Не отправляет чужие private данные

Client-side код можно разделять с сервером как библиотеку правил формата. Но сервер должен сам выполнить необходимые проверки. Совместная библиотека уменьшает дублирование исходников, а не переносит доверие в браузер.

Вернуться по памяти: Почему проверка `recipientId` в WASM не может быть единственной проверкой приватной отправки?

Для проверки деталей: [7], [8]. Ссылки на с. 39-40.

06. async и браузерный UI

await не означает отдельный поток для каждой операции.

В базовой браузерной модели UI обслуживается event loop. Blazor WASM использует браузерную среду для событий и async-операций. Когда HttpClient ждёт сеть, await позволяет вернуться к другой работе. Это не требует потока, который крутится в цикле ожидания каждого HTTP-ответа.

Когда ответ готов, продолжение C#-метода может обновить модель и инициировать рендеринг. Пока метод находится на await, пользователь способен изменить комнату, запустить другое действие или покинуть страницу. Поэтому однопоточная модель не отменяет устаревшие результаты и логические гонки.

```
var roomId = RoomId;
var data = await Http.GetFromJsonAsync<RoomResponse>(
    $"api/rooms/{roomId}");

if (RoomId != roomId) return;
_room = data;
```

Это демонстрация проверки параметра. При последовательности 1 - 2 - 1 нужен также идентификатор операции или отмена: совпадение RoomId не доказывает свежесть ответа. Этот принцип тот же, что в Blazor Server.

Долгий вычислительный цикл может блокировать отзывчивость UI. Само наличие async в сигнатуре не выносит CPU-работу в background. Не следует ожидать серверного поведения Task.Run и ThreadPool без проверки поддержки браузерной многопоточности и настроек конкретного runtime.

Web Workers и wasm threads существуют как отдельные механизмы с ограничениями и условиями размещения. Для нашего чата они не нужны. Нам достаточно не блокировать UI, ждать I/O через await и проверять актуальность операций после ожидания.

Вернуться по памяти: Как может появиться история старой комнаты, если приложение не выполняет два присваивания одновременно?

Для проверки деталей: [9], [6], [10]. Ссылки на с. 39-40.

07. Компонент и DOM: локальный рендеринг

По сети не нужно отправлять каждую переменную интерфейса.

Room.razor компилируется в тип компонента. Его экземпляр в клиентском runtime хранит поля. Разметка читает их при рендеринге. Blazor строит render tree, определяет изменения и применяет их к DOM через локальную браузерную инфраструктуру.

```
private void SelectRecipient(int userId, string userName)
{
    _recipientId = userId;
    _recipientName = userName;
}
```

В нашем чате этот метод вызывается при выборе имени. Он меняет адресата и отображение формы. Пока внутри него нет HTTP или Hub-вызова, backend не участвует в выборе. Действие может работать даже при временно потерянной связи, если UI не запретил его отдельно.

В Blazor Server аналогичный C#-метод находится на сервере, поэтому доставка события и изменения UI требует соединения. В WASM событие и рендеринг локальны. Это конкретное преимущество места выполнения, которое не зависит от того, велико ли приложение.

Изменение поля не тождественно рендерингу. После обычных событий Blazor обычно иницирует его сам. Обработчик входящего Hub-события может потребовать явного InvokeAsync и StateHasChanged для согласования с renderer. Рендеринг не читает API заново без нашего кода.

Когда backend присылает JSON сообщения, клиент сначала получает данные, затем меняет список, затем рендерит. Сервер не присылает готовый DOM-update нашего компонента. Это важное отличие прикладного SignalR-чата от UI-протокола Blazor Server.

Вернуться по памяти: Что приходит клиенту в WASM ReceiveMessage: новая разметка списка или данные сообщения? Кто формирует новый UI?

Для проверки деталей: [11], [12]. Ссылки на с. 39-40.

08. Lifecycle в standalone WASM

Параметры меняются, DOM появляется, ресурсы освобождаются.

OnInitializedAsync подготавливает экземпляр. OnParametersSetAsync реагирует на параметры. OnAfterRenderAsync выполняется после рендеринга, когда можно взаимодействовать с созданными DOM-элементами. DisposeAsync завершает принадлежащие компоненту ресурсы, например HubConnection.

В standalone WASM нет обязательной серверной prerender-фазы. Двойная инициализация из Blazor Web App с prerender не должна автоматически переноситься в объяснение нашей старой версии. Наш App запускается клиентским WebAssemblyHostBuilder.

Задача нашего Room	Подходящая точка
Загрузить комнату по RoomId	OnParametersSetAsync
Прокрутить уже показанный список	OnAfterRenderAsync
Зарегистрировать Hub handlers	До StartAsync
Завершить своё соединение	DisposeAsync

Переход /rooms/1 - /rooms/2 может сохранить экземпляр Room и изменить параметр. Поэтому мы сбрасываем получателя, typing и загружаем историю в OnParametersSetAsync. Отдельное соединение может сохраниться, а его server-group должна переключиться явно.

OnAfterRenderAsync сам по себе не означает «после каждой операции с базой». Это этап отображения. В нашем коде _scrollToLatest хранит намерение прокрутки, а реальный JS-вызов выполняется после обновления DOM. Так мы не прокручиваем ещё не существующий элемент.

Lifecycle не отменяет все async-операции автоматически. При уходе со страницы нужно остановить соединение и учитывать фоновые задержки typing. Если компонент уничтожен, поздний callback не должен продолжать бессмысленно изменять его UI.

Вернуться по памяти: Почему scrollChatToBottom вызывается после рендеринга, а не сразу после присваивания _messages?

Для проверки деталей: [13], [14]. Ссылки на с. 39-40.

09. Локальное действие или серверный сценарий?

Обработчик клика может вообще не использовать сеть.

В WASM обычная кнопка с `@onclick` связывается с локальным C#-обработчиком. Специальный компонент не требуется для отправки сообщения на backend. Сеть появляется только тогда, когда обработчик обращается к HTTP, Hub или другому сетевому API.

```
<button @onclick="SendToEveryone">Everyone</button>
@code {
    private void SendToEveryone() {
        _recipientId = null;
        _recipientName = null;
    }
}
```

Этот обработчик меняет локальное состояние. Он не сообщает серверу о смене получателя заранее. Когда пользователь отправит текст, `SendMessageToHubAsync` выберет метод и передаст нужные аргументы. Поэтому UI-выбор и серверная отправка - два разных события.

Та же кнопка может вызывать аsync-метод, в котором `HttpClient` получает данные. Тогда локальная часть начинается сразу, затем происходит `await` сети, после ответа снова продолжается клиентский код. Слово аsync не превращает весь обработчик в серверный.

Действие	Нужна сеть в нашей реализации?
Выбрать получателя	Нет
Очистить черновик	Нет
Получить историю	Да, HTTP API
Сохранить новое сообщение	Да, SignalR invocation
Отобразить полученный DTO	Локальный рендеринг

Обычная HTML-форма в браузере умеет отправить POST самостоятельно. Но в нашем Home форма использует `@onsubmit` и `preventDefault`: встроенная отправка предотвращается, а C# делает `PostAsJsonAsync`. Поэтому причина HTTP здесь находится в обработчике, а не в названии тега `form`.

Вернуться по памяти: Как проверить, отправляет ли выбор адресата запрос: по названию кнопки или по обработчику и DevTools?

Для проверки деталей: [12], [18]. Ссылки на с. 39-40.

10. @bind и события ввода

Значение DOM и поле C# связаны обработкой событий.

В нашем textarea используется @bind="_draft" и @bind:event="oninput". Browser меняет текстовое поле, возникает input-событие, Blazor передаёт новое значение в клиентскую модель. В WASM всё это локально. Затем @bind:after запускает NotifyTypingAsync.

```
<textarea @bind="_draft"
           @bind:event="oninput"
           @bind:after="NotifyTypingAsync">
</textarea>
```

Bind - удобный синтаксис для чтения значения и обработки изменения. Это не магическая общая память DOM и C#. В конкретный момент новое значение уже может быть в input, а обработка ещё не завершена. При async-работе это особенно важно.

oninput вызывается по мере изменения текста. Стандартное onchange обычно связано с завершением изменения элемента. Поэтому выбор события влияет на частоту C#-обработчика и будущих typing-уведомлений. Для каждого символа отправлять сетевой вызов не обязательно: мы ограничили частоту до одного раза в секунду.

maxlength помогает обычному вводу, но не заменяет ограничение в Domain. Значения из сети можно отправить без textarea. _draft.Trim() в SendAsync убирает крайние пробелы, проверка Length предотвращает очевидно недопустимую отправку, а сервер повторяет необходимые проверки.

Если связь с Hub оборвалась, поле и локальная модель всё ещё могут обновляться. В нашем коде NotifyTypingAsync проверяет Connected и не отправляет событие. Возможность редактировать черновик и возможность доставить typing - независимые свойства.

В клиентском приложении частые события стоят CPU и рендеринг. В Server они также создают сетевой обмен. Один синтаксис @bind имеет разную цену в разных местах исполнения.

Вернуться по памяти: Почему потеря Hub-соединения не должна автоматически стирать _draft?

Для проверки деталей: [12], [11]. Ссылки на с. 39-40.

11. Router, адрес и fallback

URL страницы - не имя физического файла на сервере.

Router внутри App выбирает компонент по @page. Адрес /rooms/2 соответствует Room.razor с параметром RoomId. Для клиентского перехода Router меняет отображаемую страницу, а программа может продолжить работать в той же вкладке без новой загрузки runtime.

При ручном открытии /rooms/2 браузер запрашивает этот адрес у сервера как документ. Сервер не хранит отдельный файл rooms/2. Ему нужно отдать оболочку index.html, после чего клиентский Router прочитает URL и выберет страницу. Для этого в старом Server был MapFallbackToFile("index.html").

```
app.MapAuthEndpoints();
app.MapRoomEndpoints();
app.MapHub<ChatHub>("/chat");
app.MapFallbackToFile("index.html");
```

Конкретные endpoints обслуживают API и Hub, fallback обслуживает подходящие неперехваченные адреса SPA. Он не означает, что backend обязан успешно распознать любую страницу. Неизвестную клиентскую страницу затем обработает Router.

Но слишком широкий fallback может маскировать неправильный адрес API: вместо ожидаемого 404 клиент получает HTML. JSON-десериализация падает, а исходная проблема - URL или маршрутизация. Поэтому при ошибке смотри статус, Content-Type и начало тела ответа.

При размещении в подпапке важны base href, абсолютные ссылки и правила nginx. Navigation.ToAbsoluteUri("chat") строит адрес относительно базы приложения. Наши ссылки /rooms/... рассчитаны на корень сайта и требуют пересмотра для подпапки.

Вернуться по памяти: Почему переход по ссылке внутри приложения может работать, а обновление страницы /rooms/2 давать серверный 404?

Для проверки деталей: [15], [16]. Ссылки на с. 39-40.

12. Один адрес, два приложения

Совместное размещение не меняет место выполнения UI.

Первоначально WASM открывался на порту 7272, backend - на 7299. Позже RealTimeChat.Server начал раздавать клиентские файлы. Пользователь получил один origin для HTML, API и Hub. Это упростило относительные адреса и работу cookie.

```
app.UseBlazorFrameworkFiles();
app.UseStaticFiles();
...
app.MapHub<ChatHub>("/chat");
app.MapFallbackToFile("index.html");
```

UseBlazorFrameworkFiles относится к нашей схеме размещения standalone клиента через ASP.NET Core. Это не универсальная инструкция для любого Blazor Web App. Статические файлы обслуживают runtime, сборки и CSS, а endpoints выполняют backend-код.

Ресурс	Кто его использует
index.html, CSS, framework files	Browser для запуска и UI
/api/rooms	HttpClient для чтения данных
/api/auth/login	HTTP-обмен входа
/chat	SignalR для событий чата

Origin состоит из схемы, host и порта. localhost:7272 и localhost:7299 - разные origins. Совпадения слова localhost недостаточно. CORS, cookies и SameSite имеют свои правила; same-origin и same-site - разные понятия.

WASM можно раздать статическим сервером или CDN, а API держать отдельно. Но тогда отдельно настраиваются адреса, CORS и аутентификация. Это может быть полезно в другом продукте, но не нужно только ради демонстрации архитектурных слоёв.

С точки зрения процессов backend выполняется на сервере, UI в браузере. Совместное размещение просто делает сеть удобнее: оно не превращает Room.razor в серверный компонент и не создаёт server-circuit.

Вернуться по памяти: Что конкретно упростилось после объединения адресов, а что осталось клиентским?

Для проверки деталей: [16], [19]. Ссылки на с. 39-40.

13. Клиентский Program и DI

Свой контейнер зависимостей внутри вкладки.

Program.cs WASM запускается в клиентском runtime. WebAssemblyHostBuilder собирает клиентскую конфигурацию, root-компоненты и DI. Это не серверный WebApplication.CreateBuilder, не запуск Kestrel и не настройка middleware backend.

```
var builder = WebAssemblyHostBuilder.CreateDefault(args);
builder.RootComponents.Add<App>("#app");
builder.RootComponents.Add<HeadOutlet>("head::after");

builder.Services.AddScoped(_ => new HttpClient
{
    BaseAddress = new Uri(builder.HostEnvironment.BaseAddress)
});

await builder.Build().RunAsync();
```

RootComponents связывает App с DOM-местом #app в index.html. HeadOutlet обслуживает содержимое head, например PageTitle. HttpClient получает BaseAddress, чтобы относительный api/rooms обращался к нужному адресному пространству.

У клиента свой DI-контейнер. Регистрация ChatService на backend не появляется в нём автоматически. Общая библиотека DTO не означает общий контейнер или общий экземпляр объекта. Каждый host регистрирует собственные зависимости.

В стандартном WASM-приложении scoped-сервисы фактически живут в scope приложения, часто всю жизнь вкладки. Не возникает нового DI scope на каждый HTTP-запрос HttpClient. Это отличается и от API scope запроса, и от server-circuit.

Клиентская конфигурация доставляется браузеру. Не кладём туда серверные пароли и приватные secrets. User Secrets старого Server оставались на сервере; они не предназначены для передачи в клиентский appsettings.

Вернуться по памяти: Почему AddScoped<HttpClient> в WASM не создаёт новый клиент при каждом GetAsync?

Для проверки деталей: [3], [17]. Ссылки на с. 39-40.

14. HttpClient в браузере

Привычный C# API использует browser networking.

В WASM HttpClient работает через браузерную инфраструктуру, в частности Fetch API. Запрос принадлежит браузеру и подчиняется его ограничениям. Это ключ к cookie: ответ login получает browser, поэтому Set-Cookie может изменить его cookie-хранилище.

```
using var response = await Http.GetAsync("api/auth/me");

if (response.StatusCode == HttpStatusCode.Unauthorized)
    return;

response.EnsureSuccessStatusCode();
```

Вызов не означает, что клиентский C# открыл обычный серверный сокет напрямую. Browser выполняет сетевую часть и возвращает runtime результат. Некоторые стандартные возможности HttpClientHandler недоступны или работают иначе, чем в обычном .NET-процессе.

Запросы к тому же origin по умолчанию используют подходящие cookie в рамках browser-правил. Для cross-origin может потребоваться BrowserRequestCredentials.Include, соответствующий CORS на backend и допустимые cookie-атрибуты. Одного Include недостаточно, если сервер или browser блокирует обмен.

CORS определяет, разрешено ли клиентскому коду данного origin читать cross-origin ответ. Это не система прав доступа к бизнес-данным. Не пытайся «обойти» CORS через NoCors: непрозрачный ответ не станет пригодным JSON для нашего приложения.

Отмена запроса через CancellationToken полезна при уходе из комнаты. Но отмена ожидания клиента не доказывает, что backend ничего не выполнил. Если POST уже обработан, запись может остаться. Эту разницу особенно важно помнить при повторной отправке.

Вернуться по памяти: Где должен появиться Set-Cookie успешного login через WASM HttpClient: в browser или в отдельном серверном HttpClient?

Для проверки деталей: [18], [19], [20]. Ссылки на с. 39-40.

15. DTO и JSON: данные вместо объектов

Через сеть не перемещается тот же экземпляр класса.

`GetFromJsonAsync<RoomResponse[]>` отправляет HTTP-запрос и десериализует ответ в клиентские объекты. Backend мог использовать `RoomInfo`, EF-сущности или другие модели внутри. Клиент получает байты JSON, а затем создаёт свои экземпляры DTO.

```
_rooms = await Http.GetFromJsonAsync<RoomResponse[]>(
    "api/rooms/") ?? [];
```

`Contracts` помогает обеим сторонам договориться о форме сообщения. Это не указатель на объект серверной памяти. Изменение `_rooms` в браузере не переименовывает `Room` в PostgreSQL. Для записи нужен отдельный сценарий на backend.

При несовпадении свойств, типов или формата может произойти ошибка десериализации. Часть лишних полей обычно игнорируется, а отсутствующие значения могут дать default; такое поведение не является гарантией совместимости любых версий. API-контракт надо менять осознанно.

Статус HTTP и тело ответа - два разных источника информации. 401 означает отсутствие допустимой аутентификации. 403 - запрет операции. 404 - ресурс или endpoint не найден. 500 - ошибка сервера. JSON-исключение после получения HTML часто указывает на неправильный маршрут или redirect.

В .NET 10 известные API-endpoints с cookie-аутентификацией обычно возвращают 401/403 вместо перенаправления. В нашем коде дополнительно использовалась настройка `DisableCookieRedirect`. Клиент всё равно должен явно обрабатывать требуемые статусы, а не считать любой ответ массивом сообщений.

Время `CreatedAt` передаётся как данные. На клиенте `ToLocalTime` использует клиентскую среду, а не часовой пояс процесса backend. Для отображения и сортировки нужен согласованный формат UTC.

Вернуться по памяти: Почему общий проект `Contracts` не убирает сериализацию и не даёт доступ к серверному объекту?

Для проверки деталей: [18], [20], [29]. Ссылки на с. 39-40.

16. Cookie-вход в нашей WASM-версии

Здесь browser получает HTTP-ответ API сам.

Home.razor показывает форму. @onsubmit вызывает клиентский LoginAsync, а preventDefault не даёт браузеру выполнить стандартную отправку HTML-формы. Метод использует PostAsJsonAsync и отправляет JSON с UserName и Password на api/auth/login.

```
using var response = await Http.PostAsJsonAsync(
    "api/auth/login",
    new LoginRequest(_userName, _password));

_password = string.Empty;
if (response.StatusCode == HttpStatusCode.Unauthorized)
{
    _loginError = "Wrong username or password.";
    return;
}
```

Backend принимает отдельный HTTP-запрос. SignInManager проверяет пароль и готовит cookie в его ответе. Browser networking получает ответ и сохраняет cookie, если правила обмена позволяют. C# затем читает CurrentUserResponse и показывает пользователя.

C# не должен извлекать защищённый билет вручную и складывать его в localStorage. Cookie управляет browser. HttpOnly запрещает обычному JavaScript читать её значение, но browser всё равно может приложить её к запросам. HttpOnly не делает сайт невосприимчивым к XSS.

После входа Home загружает комнаты. При обновлении вкладки программа стартует заново, а cookie остаётся согласно настройкам. GET api/auth/me восстанавливает UI-сведения о пользователе. Это не повторная отправка пароля.

В Blazor Server HttpClient выполнялся бы в серверном процессе, и ответ получил бы он сам. Именно поэтому одинаковое имя PostAsJsonAsync имеет разный эффект для browser-cookie в разных моделях исполнения.

Вернуться по памяти: Как изменится цепочка выдачи cookie, если перенести этот LoginAsync из WASM в интерактивный server-компонент без других правок?

Для проверки деталей: [18], [20], [21]. Ссылки на с. 39-40.

17. Состояние входа и настоящие права

CurrentUserResponse - сведения для UI, а не доказательство серверу.

После login Home хранит `_user` и показывает комнаты. При старте он вызывает `api/auth/me`. Это минимальная схема состояния UI. Она не использует полноценный `AuthenticationStateProvider` и `AuthorizeRouteView`, но может быть достаточной для первого учебного клиента.

Сервер не должен доверять `_user` из браузера. При HTTP-запросах он проверяет cookie. При установлении SignalR-соединения получает идентичность подключения. Изменение `_user.Id` в клиентской памяти не должно менять реального автора на сервере.

Что знает клиент	Что проверяет backend
Имя для отображения	Допустимый authentication ticket
Список доступных действий	Право на конкретную операцию
Выбранный RoomId	Существование и доступ к комнате
Выбранный RecipientId	Получатель и правила сообщения

При выходе наш клиент отправляет `POST api/auth/logout`. Backend возвращает ответ, удаляющий cookie, затем Home очищает `_user` и список комнат. Приложение должно также завершить или пересоздать действующие realtime-подключения согласно политике.

Удаление cookie не обязательно немедленно отзывает идентичность уже установленного WebSocket-соединения. SignalR проверяет вход при подключении; длительное соединение имеет отдельные вопросы истечения и отзыва. Есть настройки завершения при истечении и другие способы управления доступом. Это не решается одним присваиванием `_user = null`.

Если API возвращает 401, клиент должен перевести UI в состояние повторного входа и не продолжать считать все функции доступными. Для развития приложения удобно выделить общий сервис состояния аутентификации. Но такая абстракция не заменяет backend-защиту.

Вернуться по памяти: Почему очистка `_user` в Home не является серверным logout сама по себе?

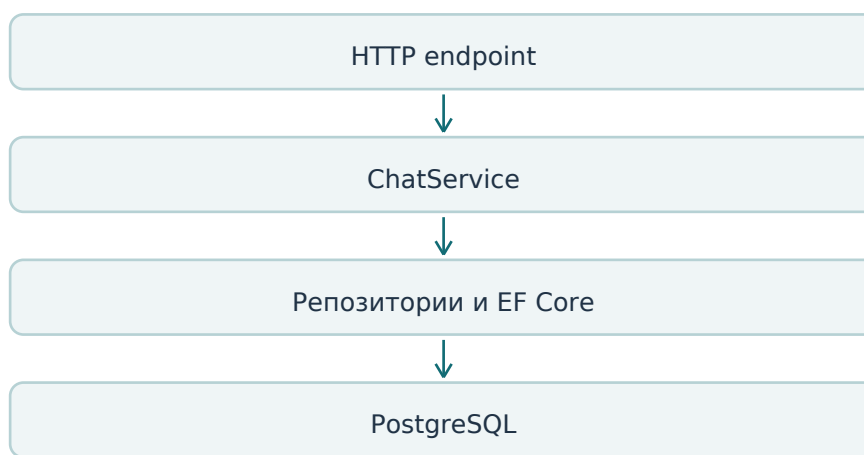
Для проверки деталей: [7], [8], [21]. Ссылки на с. 39-40.

18. Что происходит внутри endpoint

Обычная серверная обработка отдельного HTTP-запроса.

Запрос `GetRecentMessagesAsync` приходит на backend как `GET /api/rooms/{id}/messages`. `Authentication middleware` узнаёт пользователя по cookie, `authorization` проверяет доступ к endpoint. Обработчик извлекает `userId` из проверенного `principal` и вызывает `ChatService`.

`ChatService` запрашивает сообщения через репозиторий и имена авторов через `IUserRepository`. Репозиторий фильтрует `roomId` и приватную видимость, сортирует и ограничивает историю. `EF Core` переводит запрос в SQL и получает строки PostgreSQL.



В обычной HTTP-обработке `scoped DbContext` живёт в `scope` запроса. После завершения обработчика соответствующий `scope` освобождается. Клиентский `scoped HttpClient` из другой среды к этому времени жизни не относится. Одинаковое слово `scoped` описывает разные контейнеры.

Endpoint создаёт DTO и отдаёт JSON. Серверный объект не переходит в клиентский DI. В браузере десериализация создаёт новые экземпляры. Благодаря этому клиент и backend могут выполняться на разных машинах и обновляться отдельно, если контракт совместим.

Клиент отменил ожидание - сервер может получить отмену через `RequestAborted`, но не всегда успеет прекратить выполненную работу. Для GET это часто вопрос ресурсов. Для записи надо отдельно учитывать уже совершённое действие и возможный повтор.

Вернуться по памяти: Откуда endpoint берёт `userId` для фильтрации истории и почему не из произвольного клиентского параметра?

Для проверки деталей: [7], [18], [30]. Ссылки на с. 39-40.

19. Что строит HubConnectionBuilder

Build создаёт объект; StartAsync устанавливает связь.

В нашем компоненте создаётся .NET-клиент SignalR. Он знает адрес Hub, правила reconnect и обработчики событий. Это прикладное соединение чата. WASM не нуждается в нём для каждого изменения DOM: UI и без Hub выполняется локально.

```
var connection = new HubConnectionBuilder()
    .WithUrl(Navigation.ToAbsoluteUri("chat"))
    .WithAutomaticReconnect()
    .Build();
```

WithUrl задаёт endpoint. Navigation.ToAbsoluteUri связывает относительный путь с базой приложения. WithAutomaticReconnect задаёт retry-политику после потери установленной связи. Build создаёт настроенный объект, но не делает его Connected.

```
RegisterHubHandlers(connection);
await connection.StartAsync();
```

StartAsync выполняет сетевое установление: обычно negotiation, выбор транспорта и handshake Hub-протокола. WebSocket - транспорт байтов. SignalR Hub protocol - формат сообщений вызова, результата и событий поверх него. У них разные обязанности.

При одном origin browser может приложить подходящую cookie к установлению соединения. Backend проверяет Authorize на Hub. Неудачный вход, запрещённый origin, неправильный адрес или недоступный transport мешают старту. Проверять только наличие объекта _hubConnection недостаточно.

Handlers регистрируются до StartAsync, чтобы клиент был готов к входящим событиям. Регистрация ReceiveMessage сама не читает историю и не вступает в комнату. После успешного старта мы отдельно вызываем JoinRoom.

StartAsync может завершиться ошибкой при первом подключении. WithAutomaticReconnect не превращает начальную ошибку StartAsync в бесконечные автоматические попытки.

Вернуться по памяти: Что уже существует после Build и чего ещё нет? Какие два шага нужны до получения событий своей комнаты?

Для проверки деталей: [22], [27]. Ссылки на с. 39-40.

20. Вызов Hub и входящее событие

invoke и on смотрят в разные направления.

InvokeAsync("SendMessage", roomId, text) просит backend выполнить метод Hub. Клиент сериализует имя, аргументы и сведения о вызове. Сервер выбирает метод, проверяет и выполняет его, затем возвращает completion или ошибку. Это не обычный локальный вызов C#-объекта ChatHub.

```
await connection.InvokeAsync(
    "SendMessage", roomId, text);

connection.On<ChatMessageResponse>(
    "ReceiveMessage", HandleReceivedMessageAsync);
```

On регистрирует callback для сообщения от сервера. В нашем Hub после сохранения выполняется Clients.Group(...).SendAsync("ReceiveMessage", message). Событие доставляется всем выбранным клиентам, включая автора broadcast-сообщения. Клиент получает DTO и добавляет его в UI.

Completion InvokeAsync и ReceiveMessage - разные сообщения протокола. Их нельзя считать одним ответом. Сервер может уже отправить событие клиенту, а завершение invocation ещё доставляется; порядок нужно понимать по конкретному коду и обработчикам.

Наш SendMessage не возвращает ChatMessageInfo как return value: он сохраняет и публикует событие. Поэтому SendAsync после успешного invocation очищает черновик, а отображение сообщения выполняет handler ReceiveMessage. Если дополнительно вставлять сообщение оптимистично, потребуются ID и политика дедупликации.

Имя события - договорённость. ReceiveMessage не является магическим встроенным событием SignalR. Тип аргумента тоже должен соответствовать данным. В нашем backend ChatMessageInfo и клиентском ChatMessageResponse совпадают по форме JSON, хотя являются разными типами.

Вернуться по памяти: Почему await InvokeAsync("SendMessage") и handler ReceiveMessage не стоит считать одним и тем же callback?

Для проверки деталей: [22], [23]. Ссылки на с. 39-40.

21. Hub: экземпляр, соединение, пользователь

Три разные идентичности.

SignalR создаёт экземпляры Hub для обработки вызовов. Это короткоживущий объект, а не вечный ChatHub на всё приложение или весь срок соединения. Нельзя хранить текущее roomId только в обычном поле Hub и ожидать, что следующий вызов увидит то же значение.

Context.ConnectionId идентифицирует подключение. Context.UserIdentifier идентифицирует пользователя по выбранной схеме, обычно через claim NameIdentifier. Один пользователь может иметь несколько подключений из разных вкладок. Поэтому connectionId и userId нельзя подменять друг другом.

Сущность	Где держим нужное состояние
Экземпляр ChatHub	Зависимости одного вызова
Подключение	Context.Items и групповые связи
Пользователь	Проверенный principal / UserIdentifier
Присутствие приложения	Общий RoomPresence

Наш GetCurrentUserId читает Context.UserIdentifier, преобразует в int и не принимает автора из сообщения клиента. CurrentRoomKey лежит в Context.Items и связывает подключение с текущей комнатой. RoomPresence имеет singleton lifetime и хранит участия всех соединений процесса.

Для нашего ChatService с scoped репозиториями Hub-вызов создаёт подходящую границу разрешения зависимостей. Не надо переносить этот сервис в поле singleton RoomPresence. Состояние присутствия и операции БД имеют разные сроки жизни.

OnDisconnectedAsync вызывается при обнаруженном закрытии соединения. Это не мгновенный достоверный сигнал «человек выключил компьютер». Framework должен обнаружить потерю, а при сетевом сбое есть задержки. Presence требует разумной интерпретации событий.

Вернуться по памяти: Почему поле _currentRoomId в ChatHub ненадёжно для следующего вызова и где наш код хранит комнату вместо него?

Для проверки деталей: [23], [8]. Ссылки на с. 39-40.

22. Комнаты и группы SignalR

Группа - механизм адресации, а не правило доступа.

Наш Hub использует группы вида `room:1`. `JoinRoom` проверяет существование комнаты, узнаёт прежнюю комнату подключения, удаляет старую `group membership` и добавляет новую. Затем обновляет `Context.Items` и `RoomPresence` и публикует `online`-списки.

```
await Groups.AddToGroupAsync(
    Context.ConnectionId, $"room:{roomId}");

await Clients.Group($"room:{roomId}")
    .SendAsync("ReceiveMessage", message);
```

Добавление в группу влияет на будущую доставку сообщений этого подключения. Оно не читает PostgreSQL, не загружает старые сообщения и не доказывает право доступа к любой комнате. Проверки доступа выполняет приложение до добавления.

Клиентский `RoomId` тоже не является доказательством `membership`. Наш `EnsureJoinedRoom` сравнивает переданный `roomId` с `Context.Items`. Поэтому ручная попытка отправить в старую комнату после `switch` отклоняется. Нельзя защитить это только отключённой кнопкой UI.

При обычном новом соединении членство старого подключения не следует считать сохранённым. Поэтому после `reconnect` клиент повторно вызывает `JoinRoom`. Наличие отдельной `stateful reconnect`-настройки меняет некоторые свойства восстановления, но наш `WithAutomaticReconnect` сам по себе её не включает.

SignalR groups удобны для доставки, но не являются полноценной базой `presence`: API не предназначен для универсального надёжного подсчёта всех пользователей группы. Для списка онлайн мы храним явные участия в `RoomPresence`.

Повторный `JoinRoom` должен иметь определённое поведение. Наш код возвращается при совпадении текущей комнаты. При изменении сценариев восстановления проверь, получает ли клиент актуальное `presence`-состояние.

Вернуться по памяти: Почему авторизация должна происходить до `AddToGroupAsync`, а не выводиться из факта `membership`?

Для проверки деталей: [24], [8], [27]. Ссылки на с. 39-40.

23. Приватная доставка

Не отправлять данные лишнему клиенту важнее, чем скрыть их.

В нашем ChatHub `SendMessage` сохраняет сообщение с `RecipientId` и отправляет `ReceiveMessage` через `Clients.Users` автору и получателю. Список адресатов дедуплицируется, если они совпали. Это адресация по пользователям, а не только по одному `connectionId`.

```
var userIds = new[]
{
    message.AuthorId.ToString(),
    recipientId.ToString()
}.Distinct().ToArray();

await Clients.Users(userIds)
    .SendAsync("ReceiveMessage", message);
```

`Users` может доставить событие нескольким вкладкам того же пользователя. Наш handler проверяет `RoomId` и не добавляет сообщение в UI другой комнаты. Значит private-событие может прийти в подключение пользователя вне текущего room-экрана, но данные всё равно получает разрешённый пользователь.

Если модель продукта требует отправлять только активным участникам конкретной комнаты, нужна более узкая адресация и дополнительные серверные проверки. Текущий учебный код проверяет существование получателя, но не означает автоматически, что получатель онлайн или находится в той же комнате.

История тоже фильтруется на backend: public-сообщения, свои private или адресованные текущему пользователю. Нельзя вернуть полный список всем клиентам, а затем отфильтровать его LINQ в браузере. Скрытые данные уже были бы у пользователя.

Имя `RecipientName` удобно для отображения, но доступ определяется ID и проверенной идентичностью. Совпадение `nickname` не является надёжной границей приватности. В новом дизайне правила можно расширить, но их надо формулировать явно.

Вернуться по памяти: Чем различаются «клиент не показывает сообщение другой комнаты» и «backend не передаёт сообщение чужому пользователю»?

Для проверки деталей: [24], [8], [7]. Ссылки на с. 39-40.

24. История и live-события

Два источника требуют согласования.

В нашем `OnParametersSetAsync` сначала загружается история API, затем устанавливается соединение и вызывается `JoinRoom`. Есть промежуток: сообщение может сохраниться после снимка истории, но до `group membership`. Оно не попадёт ни в ответ истории, ни в live-доставку этого подключения.

Это реальное ограничение учебной последовательности, а не гарантия SignalR. Простое улучшение - подписаться и вступить в комнату, затем получить историю и объединить её с уже пришедшими событиями. Но здесь нужно аккуратно буферизовать и дедуплицировать, чтобы поздний ответ не затёр live-сообщения.

```
_messages = _messages
    .Concat(recentMessages)
    .DistinctBy(message => message.Id)
    .OrderBy(message => message.CreatedAt)
    .ThenBy(message => message.Id)
    .ToArray();
```

ID помогает узнавать одно и то же сохранённое сообщение. Сортировка по времени и затем ID даёт устойчивое отображение в нашей схеме. Это не универсальное доказательство глобального порядка всех distributed-вызовов. Для строгих гарантий нужна явно спроектированная последовательность.

После `await` надо проверить, что операция относится к текущей комнате. В `HandleReceivedMessageAsync` мы проверяем `RoomId` и дубликат до JS-вызова, затем повторяем проверку после него. Пока ожидался JS, другое событие могло добавить сообщение или пользователь мог перейти.

`GetRecent` возвращает только двадцать записей. Если за разрыв прошло больше сообщений, объединение `latest 20` не восстановит всё. Для полной синхронизации нужен `cursor`, например `lastSeenMessageId`, и endpoint «получить после ID» с нужной фильтрацией.

Вернуться по памяти: Назови окно потери в первоначальном порядке «история, затем `JoinRoom`». Как ID помогает при альтернативном порядке?

Для проверки деталей: [18], [22]. Ссылки на с. 39-40.

25. Reconnect: восстановить транспорт и состояние

Успешное соединение ещё не означает готовую комнату.

WithAutomaticReconnect без параметров задаёт интервалы попыток 0, 2, 10 и 30 секунд после потери установленного соединения. Это задержки между попытками, а не обещание вернуть связь за определённое суммарное время. При неудаче всех попыток клиент переходит в Disconnected и вызывает Closed.

Reconnecting позволяет отключить отправку и показать состояние. Reconnected означает, что связь снова установлена. Наш RestoreRoomAfterReconnectAsync затем выполняет JoinRoom, читает свежую историю и объединяет её с локальными сообщениями. Только после этого выставляет «Connected to the room».

```
connection.Reconnecting += _ =>
    SetConnectionStatusAsync("Reconnecting...");

connection.Reconnected += _ =>
    RestoreRoomAfterReconnectAsync(connection);

connection.Closed += _ =>
    SetConnectionStatusAsync("Disconnected");
```

Начальная ошибка StartAsync не покрывается этой политикой автоматически. Если при первом открытии сервер недоступен, нужны отдельная попытка запуска, кнопка retry или другая выбранная стратегия. Не следует обещать бесконечный reconnect от одной настройки.

Транспорт может стать Connected, а восстановление JoinRoom завершиться ошибкой. Это разные состояния. Удобно отдельно хранить готовность комнаты, чтобы кнопка Send зависела не только от HubConnectionState.Connected. Серверная EnsureJoinedRoom остаётся обязательной проверкой.

Автоматическое восстановление не гарантирует получение всех пропущенных сообщений. Наш API last 20 покрывает небольшой разрыв. Для длинного разрыва нужен cursor; для подтверждённой записи при потере ответа - политика повторов и идиempотентность.

Вернуться по памяти: Какие три действия нужны после обычного reconnect, чтобы UI можно было считать восстановленным?

Для проверки деталей: [22], [27]. Ссылки на с. 39-40.

26. Online users: считать участия

Один человек может иметь несколько соединений.

RoomPresence хранит ConcurrentDictionary с ключом connectionId и значениями roomId, userId, userName. Join меняет участие, Leave удаляет его. GetUsers фильтрует по комнате и делает DistinctBy(userId), чтобы показать каждого человека один раз.

Две вкладки одного пользователя дают две записи участия, но одно имя в списке. При закрытии одной записи другая остаётся. Поэтому хранить только HashSet userId и удалять пользователя при любом disconnect было бы ошибкой.

```
var participants = _roomParticipants.Values
    .Where(p => p.roomId == roomId)
    .DistinctBy(p => p.userId);
```

ConcurrentDictionary защищает отдельные операции доступа к коллекции. Он не превращает несколько независимых действий Join, смены groups и публикации в одну атомарную транзакцию. Между ними могут быть изменения других подключений. Для более строгого presence нужно определить порядок и синхронизацию.

Hub публикует OnlineUsersChanged(roomId, users). Клиентский handler проверяет roomId, присваивает новый массив и инициирует рендеринг. Это снимок списка на момент публикации, а не запрос клиента на каждый показ.

Disconnect обнаруживается не обязательно мгновенно. Сеть, timeout и reconnect влияют на задержку. Поэтому online означает «backend считает участие живым», а не физическую гарантию, что человек прямо сейчас смотрит в экран.

Сервис живёт в памяти одного процесса. После перезапуска список строится заново через подключения. Это нормально для присутствия, но отличается от долговечной истории PostgreSQL. Для нескольких процессов требуется общий дизайн presence.

Вернуться по памяти: Почему DistinctBy нужен при отображении, но не должен уничтожать отдельные записи участия?

Для проверки деталей: [23], [24], [27]. Ссылки на с. 39-40.

27. Typing: событие с коротким сроком

Таймеры могут устаревать так же, как ответы API.

При изменении `_draft` наш `NotifyTypingAsync` проверяет наличие текста, `Connected` и время последнего уведомления. Ограничение одного события в секунду уменьшает нагрузку. Hub проверяет `membership` и отправляет `UserTyping` остальным в группе.

Клиент получает `roomId`, `userId` и имя. Показывает индикатор и увеличивает `_typingSequence`. Затем запускает задержку на три секунды. После задержки проверяет, относится ли таймер к текущей комнате и последней серии уведомлений.

```
if (roomId != RoomId || sequence != _typingSequence)
    return;

_typingUserName = null;
StateHasChanged();
```

Если за время ожидания пришло новое `typing`-событие, старый таймер не должен скрывать свежий индикатор. Счётчик `sequence` отличает поколения состояния. Это та же идея, что версия загрузки комнаты после `await`.

`Typing` не сохраняется как сообщение в PostgreSQL. Это временное UI-событие, потеря которого обычно допустима. Отсутствие новых событий и тайм-аут убирают индикатор. Такая модель проще, чем обязательная отправка «перестал печатать» при каждом возможном закрытии вкладки.

Учебный компонент хранит одно имя `typing`, поэтому одновременные авторы заменяют друг друга. Для нескольких индикаторов нужен словарь с отдельными сроками. Также полезна отмена задержек при `DisposeAsync`: обработка исключений в `fire-and-forget` не равна полному управлению жизнью задачи.

Доставка `typing` не обязана иметь те же гарантии, что сохранение сообщения. Надёжность выбирают по смыслу данных.

Вернуться по памяти: Почему таймер без `sequence` может скрыть индикатор пользователя, который продолжает печатать?

Для проверки деталей: [22], [9], [14]. Ссылки на с. 39-40.

28. JS interop и прокрутка

DOM измеряет browser; C# выбирает поведение.

Наш компонент хранит `ElementReference` на область сообщений. Через JS вызывает `isChatAtBottom`, а после рендеринга - `scrollChatToBottom`. Это пример разделения: C# решает, нужна ли прокрутка, JavaScript читает и меняет свойства реального DOM-элемента.

```
var wasAtBottom = await JS.InvokeAsync<bool>(
    "isChatAtBottom", _messageScroll);

_messages = [.. _messages, message];
_scrollToLatest = wasAtBottom;
```

Если пользователь читает старые сообщения, новое сообщение не должно насильно переносить его вниз. Поэтому перед изменением списка измеряем положение. После нового рендеринга проверяем `_scrollToLatest` и выполняем прокрутку, когда DOM уже соответствует новому списку.

JS interop в WASM обычно локальный. В Server такой вызов требует доставки браузеру и возврата результата через соединение. Один и тот же `InvokeAsync` имеет разную стоимость, поэтому частые мелкие DOM-вызовы стоит оценивать.

Мы повторно проверяем `RoomId` и дубликат после ожидания `isChatAtBottom`. Это не паранойя: пока `await` не завершён, другое событие может изменить состояние компонента. Измеренный результат тоже может устареть, если пользователь прокрутил страницу в промежутке.

Не изменяй через сторонний JavaScript DOM-узлы, которыми Blazor управляет, без продуманной границы. Framework рассчитывает на согласованное дерево. Измерение и изменение `scrollTop` обычно не вмешивается в структуру, а произвольное удаление узлов может нарушить `renderer`.

Для больших интеграций лучше JS-модуль и явное освобождение ресурсов. Несколько глобальных функций достаточны для нашего небольшого учебного `scroll-helper`.

Вернуться по памяти: Почему недостаточно вызвать `scrollChatToBottom` сразу после добавления объекта в `_messages`?

Для проверки деталей: [25], [13], [11]. Ссылки на с. 39-40.

29. Ownership, отмена и освобождение

Кто создал ресурс, тот должен определить его жизнь.

Наш Room создаёт HubConnection и реализует IAsyncDisposable. При уходе со страницы DisposeAsync завершает соединение. Это соответствует ownership компонента. Если позже сделать общий scoped connection-сервис на всё приложение, обязанность завершения изменится: нельзя закрывать общее подключение при любом уходе из одной страницы.

```
public async ValueTask DisposeAsync()
{
    if (_hubConnection is not null)
        await _hubConnection.DisposeAsync();
}
```

Регистрация On возвращает подписку, которую можно освобождать. Если соединение продолжает жить, а компонент ушёл, его handlers должны быть сняты. Иначе события могут удерживать старую страницу и вызывать её callbacks. При уничтожении собственного connection ситуация проще, но ownership всё равно надо понимать.

HTTP-загрузки можно отменять токеном при смене комнаты или Dispose. Таймеры typing тоже имеют жизнь. Отмена ожидания не всегда мгновенно прекращает backend, а поздние результаты следует проверять на актуальность.

Transient и scoped в клиентском DI не означают ресурс «на один клик». В стандартной вкладке scoped долго живёт. Общие сервисы не должны бесконтрольно накапливать component callbacks. Коллекции сообщений тоже могут расти, если после initial last 20 добавлять все события без ограничения.

Ошибка disposal не должна скрывать основную проблему сценария. Логи полезны, но нельзя просто проглотить все исключения и считать работу корректной. Разделяй ожидаемый уход пользователя, сетевую потерю и программную ошибку.

Вернуться по памяти: Почему перенос HubConnection из компонента в общий сервис требует пересмотра DisposeAsync и всех подписок?

Для проверки деталей: [14], [17], [22]. Ссылки на с. 39-40.

30. Размер, производительность и offline

WASM даёт возможность, но не готовый режим работы без сети.

Клиентская загрузка включает runtime и зависимости. Для размера важны Release-публикация, trimming, сжатие, lazy loading и разумный набор библиотек. Для скорости после старта - алгоритмы, рендеринг, DOM, interop и при необходимости AOT. Эти оптимизации измеряют по конкретному узкому месту.

Наш чат не становится offline-приложением только потому, что компоненты локальны. Уже загруженный UI может продолжать редактировать черновик, но новый запуск без сети требует доступных кэшированных ресурсов. История, сохранение и доставка сообщений требуют отдельно организованного локального и серверного поведения.

PWA использует service worker для ресурсов и других возможностей. Стандартный offline-кэш не превращает автоматически ответы API в локальную БД и не создаёт очередь отправки. Для offline-сообщений нужны правила хранения, повторов, конфликтов и видимости данных.

Что хотим ускорить	Что измерять
Первое открытие	Размер и время загрузки ресурсов
Редактирование большой таблицы	CPU и локальный рендеринг
Получение истории	API, БД, сеть и JSON
Длинный список сообщений	DOM, память, виртуализация
Возврат после обрыва	Reconnect и синхронизация данных

Наш `_messages` растёт после начальной загрузки. Для долгой сессии можно ограничить окно или виртуализировать список. Но виртуализация меняет измерение прокрутки и поведение высот: её нельзя считать бесплатной заменой любого `ul`.

Для маленького чата сначала важнее корректный reconnect и отсутствие потерь, чем AOT и сложная offline-очередь. Учебная версия позволяет видеть эти направления без обязательного внедрения всех сразу.

Вернуться по памяти: Чего не хватает обычному WASM-клиенту, чтобы перезапуститься без сети и позже надёжно отправить накопленные сообщения?

Для проверки деталей: [4], [6], [26]. Ссылки на с. 39-40.

31. Две архитектуры одного чата

Сравнить цену конкретного действия.

В WASM UI локален. История идёт через API, realtime - через наш ChatHub. В Server UI хранится на сервере, компонент вызывает Application напрямую, а notifier обновляет другие компоненты. Браузер получает изменения интерфейса через framework.

Действие	WASM	Server
Выбор адресата	Локальный C#	Событие к серверу
Чтение истории	HTTP API и JSON	Прямой сервисный вызов
Отправка текста	Invoke в ChatHub	Серверный сценарий
Приём сообщения	DTO и локальный рендер	Notifier и UI-update
UI-состояние	Память browser-runtime	Память circuit на сервере

Сложность WASM в нашем чате частично обусловлена явными границами: статусы API, контракт, cookie в browser, соединение, восстановление и согласование истории. Server убирает часть этих границ, но добавляет долгие DI scopes, состояние circuits и зависимость событий от сетевой задержки.

WASM полезен, когда локальные действия и состояние существенны: редакторы, сложные таблицы, черновики и работа при нестабильной сети. Server удобен для прямых серверных сценариев и ограниченной аудитории со стабильной связью. Для каталога может подойти обычный SSR.

Ни один вариант не отменяет серверную авторизацию, долговечное хранение и проектирование realtime. «Enterprise» не является техническим требованием. В roadmap WASM - также учебная возможность повторить технологию на богатом интерфейсе.

Наш новый Server-этап не доказывает, что WASM был ошибкой. Он позволяет сравнить, какой код исчезает при переносе UI на сервер и какие обязанности появляются взамен.

Вернуться по памяти: В какой модели локальный выбор получателя возможен без связи? Где за это заплачено более сложной границей с backend?

Для проверки деталей: [1], [7]. Ссылки на с. 39-40.

32. Debugger, DevTools и deploy

Разделить местную проблему и сетевую.

В DevTools Network сначала проверь документ и framework-ресурсы. Затем `api/auth/me`, `rooms` и сообщения. Затем `negotiation` и `transport` Hub. Статус 200 у `index.html` не доказывает, что приложение загрузилось и подключилось к чату.

Для локального действия поставь breakpoint в `SelectRecipient`. Для сетевого - в `SendAsync` клиента и `SendMessage` Hub на backend. Между ними есть сериализация и протокол. Отладка клиентского .NET использует browser tooling и поддержку IDE; backend debugger относится к другому процессу.

Симптом	Что проверить сначала
Прямой <code>/rooms/1</code> даёт 404	SPA fallback и base href
JSON parse error	Content-Type и тело ответа
API 401	Cookie и актуальный вход
Hub не стартует	Адрес, negotiation, auth, transport
Connected, но send запрещён	Восстановление JoinRoom
Дубликат после reconnect	ID и merge истории

При deploy nginx должен обслужить SPA-маршруты и пропустить Hub-транспорт, а backend - правильно учитывать прокси и HTTPS. Для одного origin относительные адреса проще, но всё равно важны пути и cookie-настройки. Не отдавай опубликованные секреты в клиентские assets.

Cache может создать несовместимые версии bundle и backend. Применяй согласованную публикацию и учитывай cache-policy, а для PWA - обновление service worker. Долго открытая вкладка может продолжать работать со старым DTO-контрактом.

После исправления проверяй именно нарушенный путь. Для reconnect - обрыв установленного соединения, для initial start - недоступность сервера до входа. Это разные эксперименты. Затем зафиксируй причину и рабочее изменение коммитом.

Вернуться по памяти: Как доказать, что проблема не в локальном рендеринге, а в том, что `ReceiveMessage` вообще не пришёл?

Для проверки деталей: [16], [22], [28]. Ссылки на с. 39-40.

Практика 1. Где выполняется C#?

От локального клика к двум разным сетевым механизмам.

А. Запуск

Открой WASM-этап в отдельном checkout по тегу dotnet-chat-wasm-v4.0.0. В DevTools включи Preserve log. Полностью загрузи страницу. Раздели запросы на документ, framework-ресурсы, клиентские assets и API. Найди момент, после которого начал выполняться `Home.OnInitializedAsync`.

Б. Локальное событие

В комнате выбери адресата из списка. До запуска предскажи, появится ли HTTP или Hub-вызов. Поставь breakpoint в `SelectRecipient` и посмотри, какие поля меняются. Затем отключи сеть в DevTools и повтори выбор: отдели локальную возможность от разрешённой отправки.

В. HTTP

Верни сеть. Найди `api/rooms/{id}/messages` и тело JSON. Поставь breakpoint в endpoint backend и в продолжении клиентского `GetFromJsonAsync`. Нарисуй два объекта: server DTO перед сериализацией и client DTO после десериализации. Объясни, почему это разные экземпляры.

Г. SignalR

Отправь сообщение. Найди Hub-транспорт и время вызова `SendAsync`. Поставь breakpoint в `ChatHub.SendMessage` и `HandleReceivedMessageAsync` клиента. Найди отдельные роли invocation completion и `ReceiveMessage`. Не считай каждый frame отдельным пользовательским действием.

Д. UI после данных

Посмотри, когда сообщение появляется в `_messages` и когда в DOM. Вспомни путь render tree - изменения - DOM. Объясни, почему сервер не обязан присылать HTML для нового сообщения.

Не вставляй значения cookie и паролей в отчёт. Для понимания достаточны адрес, метод, статус, тип данных и последовательность событий.

Вернуться по памяти: Через день восстанови путь одного Send без исходников: UI, HubConnection, backend, БД, событие, renderer.

Практика 2. Восстановление и гонки

Тестируем то, чего не видно при спокойной работе.

1. Начальный старт и reconnect

Сначала открой приложение при недоступном backend. Затем отдельно оборви уже установленное соединение. Сравни поведение: какая ошибка относится к StartAsync, а какая запускает WithAutomaticReconnect? До опыта запиши ожидаемые callbacks.

2. Пропущенные сообщения

Во время разрыва отправь сообщения из другого клиента. Восстанови связь. Проверь JoinRoom и merge истории по ID. Затем повтори с количеством сообщений больше двадцати. Объясни, почему last 20 не является гарантией восстановления всех пропусков.

3. Switch во время ожидания

Замедли сеть в DevTools. Быстро перейди 1 - 2 - 1. Добавь лог версии загрузки и RoomId, не меняя результат заранее. Проверь, способен ли старый ответ перезаписать более свежую историю. Предложи отмену или version check и объясни их границы.

4. Presence в двух вкладках

Открой две вкладки одного вошедшего пользователя. Сравни UserIdentifier и ConnectionId. Закрой одну. Почему пользователь должен остаться в online-списке? Затем оборви сеть и измерь, когда backend обнаруживает disconnect.

5. Privates

Используй три разных пользователя. Автор отправляет private получателю. Третий не должен получать сообщение ни в history API, ни в Hub-событиях. Проверь сеть и backend-предикат, а не только отсутствие карточки в UI.

Каждый найденный дефект формулируй: действие, ожидание, фактическое поведение и причина. Исправление проверяй тем же сценарием. После цельного рабочего исправления делай commit.

Вернуться по памяти: Какая из этих проверок выявляет логическую гонку без настоящей многопоточности?

Самопроверка: опорные ответы

Сначала собственное объяснение, затем сверка.

Место исполнения. В standalone WASM компоненты и клиентский Program выполняются в browser-runtime. Backend раздаёт файлы и отдельно обслуживает API и ChatHub. Общий адрес не переносит UI на сервер.

WASM и IL. .NET runtime работает в WebAssembly и может исполнять IL через интерпретатор с оптимизациями. AOT заранее компилирует код в wasm. Расширение ресурса .wasm не доказывает AOT приложения: managed-assembly может быть упакована в Webcil.

HTTP и cookie. Клиентский HttpClient использует browser networking. Именно browser получает Set-Cookie ответа login и хранит cookie. В server-компоненте аналогичный HttpClient выполняется на сервере, поэтому цепочка другая.

DI. У клиента и backend разные контейнеры. Scoped в WASM обычно живёт всю жизнь приложения вкладки. Scoped DbContext endpoint принадлежит обработке HTTP-запроса на backend.

HubConnection. Build создаёт объект. StartAsync устанавливает transport и Hub protocol. JoinRoom отдельно настраивает прикладную membership. WithAutomaticReconnect восстанавливает потерянную установленную связь, но не исправляет начальную ошибку StartAsync автоматически.

invoke и on. InvokeAsync вызывает серверный метод и ждёт completion. On регистрирует handler события от сервера. ReceiveMessage - наше название, не встроенная сущность SignalR. Клиент получает DTO и сам рендерит UI.

История. API и live-поток могут перекрываться или иметь окно пропуска. Нужны согласованный порядок, merge и ID. Latest 20 после reconnect не обеспечивает полную синхронизацию длинного разрыва.

Presence и typing. Presence считает отдельные подключения и показывает уникальных пользователей. Typing имеет ограниченный срок, а sequence защищает от старого таймера. Эти данные не обязаны иметь такую же долговечность, как сообщения.

Безопасность. Backend устанавливает автора и фильтрует private-данные. Клиентские проверки нужны для UX, но не являются границей доступа. CORS не заменяет авторизацию.

Сравнение с первым учебником: Server передаёт UI-события и render updates; наш WASM-чат передаёт API-запросы и прикладные сообщения. В обоих вариантах DOM находится в браузере.

Источники: browser и компоненты

Официальная документация, browser API и наш исходный код.

Номера внизу разделов отсылают к этим ссылкам. Выбирай .NET 10 в документации. Для standalone WASM и Interactive WebAssembly в Blazor Web App настройки различаются: проверяй, какую модель описывает пример.

[1] Модели размещения Blazor

<https://learn.microsoft.com/en-us/aspnet/core/blazor/hosting-models?view=aspnetcore-10.0>

[2] Структура проектов Blazor

<https://learn.microsoft.com/en-us/aspnet/core/blazor/project-structure?view=aspnetcore-10.0>

[3] Запуск и bootstrap

<https://learn.microsoft.com/en-us/aspnet/core/blazor/fundamentals/startup?view=aspnetcore-10.0>

[4] WebAssembly build tools и AOT

<https://learn.microsoft.com/en-us/aspnet/core/blazor/webassembly-build-tools-and-aot?view=aspnetcore-10.0>

[5] Упаковка Webcil: раздел руководства по размещению

<https://learn.microsoft.com/en-us/aspnet/core/blazor/host-and-deploy/webassembly/?view=aspnetcore-10.0#webcil-packaging-format-for-net-assemblies>

[6] Производительность WASM-runtime

<https://learn.microsoft.com/en-us/aspnet/core/blazor/performance/webassembly-runtime-performance?view=aspnetcore-10.0>

[7] Безопасность Blazor и доверие к клиенту

<https://learn.microsoft.com/en-us/aspnet/core/blazor/security/?view=aspnetcore-10.0>

[8] Аутентификация и авторизация SignalR

<https://learn.microsoft.com/en-us/aspnet/core/signalr/authn-and-authz?view=aspnetcore-10.0>

[9] Synchronization context Blazor

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/synchronization-context?view=aspnetcore-10.0>

[10] Browser execution model

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model

[11] Рендеринг компонентов

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/rendering?view=aspnetcore-10.0>

[12] События и обработчики

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/event-handling?view=aspnetcore-10.0>

[13] Lifecycle компонентов

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/lifecycle?view=aspnetcore-10.0>

[14] Освобождение компонентов

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/component-disposal?view=aspnetcore-10.0>

[15] Router и навигация

<https://learn.microsoft.com/en-us/aspnet/core/blazor/fundamentals/routing?view=aspnetcore-10.0>

Источники: API и realtime

Официальная документация, browser API и наш исходный код.

Документация уточняет детали API. Разбор нашего конкретного кода основан на присланных исходниках WASM-этапа. Ограничения с history-gap и last 20 являются выводами из его последовательности, а не обещаниями протокола SignalR.

[16] Размещение WASM

<https://learn.microsoft.com/en-us/aspnet/core/blazor/host-and-deploy/webassembly?view=aspnetcore-10.0>

[17] Dependency injection

<https://learn.microsoft.com/en-us/aspnet/core/blazor/fundamentals/dependency-injection?view=aspnetcore-10.0>

[18] HttpClient, Fetch и Web API

<https://learn.microsoft.com/en-us/aspnet/core/blazor/call-web-api?view=aspnetcore-10.0>

[19] CORS в ASP.NET Core

<https://learn.microsoft.com/en-us/aspnet/core/security/cors?view=aspnetcore-10.0>

[20] Cookie authentication

<https://learn.microsoft.com/en-us/aspnet/core/security/authentication/cookie?view=aspnetcore-10.0>

[21] Standalone WASM и Identity

<https://learn.microsoft.com/en-us/aspnet/core/blazor/security/webassembly/standalone-with-identity/?view=aspnetcore-10.0>

[22] SignalR .NET client и reconnect

<https://learn.microsoft.com/en-us/aspnet/core/signalr/dotnet-client?view=aspnetcore-10.0>

[23] SignalR Hubs

<https://learn.microsoft.com/en-us/aspnet/core/signalr/hubs?view=aspnetcore-10.0>

[24] Пользователи и группы SignalR

<https://learn.microsoft.com/en-us/aspnet/core/signalr/groups?view=aspnetcore-10.0>

[25] JS interop

<https://learn.microsoft.com/en-us/aspnet/core/blazor/javascript-interoperability/?view=aspnetcore-10.0>

[26] PWA и offline-ресурсы

<https://learn.microsoft.com/en-us/aspnet/core/blazor/progressive-web-app/?view=aspnetcore-10.0>

[27] Конфигурация транспорта и stateful reconnect

<https://learn.microsoft.com/en-us/aspnet/core/signalr/configuration?view=aspnetcore-10.0>

[28] Размещение WASM через nginx

<https://learn.microsoft.com/en-us/aspnet/core/blazor/host-and-deploy/webassembly/nginx?view=aspnetcore-10.0>

[29] JSON и HttpClient

<https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/httpclient-extensions>

[30] Lifetime и работа EF Core

<https://learn.microsoft.com/en-us/ef/core/dbcontext-configuration/>

[31] Изменения bootstrap в .NET 10

<https://learn.microsoft.com/en-us/aspnet/core/release-notes/aspnetcore-10.0?view=aspnetcore-10.0>